Contents

1. Graph Algorithms

This chapter covers graph algorithm problems. For foundational concepts, see the Concepts section: **What is a Graph?** for graph representations, **DFS** for Depth-First Search, **BFS** for Breadth-First Search, **Dijkstra's Algorithm** for weighted shortest paths, and **DSU** for Disjoint Set Union.

1.1. Counting Rooms

Question - Labyrinth Backup Link

Explanation :

Concepts used: **DFS** (Depth-First Search) - see Concepts

A room is just a group of floor squares that are connected. You can move between them up, down, left, or right. Our goal is to count how many separate rooms exist. Here's the algorithm:

1. Scan each cell in the grid.
2. If you find an unvisited floor cell (.), that means you've discovered a new room.
3. Run a DFS from that cell: move to all connected floor cells, marking them as visited.
4. Continue scanning the grid. Every time you start a new DFS, that's a new room.

At the end, the number of DFS calls equals the number of rooms.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int n, m, rooms = 0;
vector<string> grid;
vector<vector<bool>> visited;

// Movement in 4 directions: right, left, down, up
int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0};

// Check if a cell is inside the grid, unvisited, and is floor
bool isValid(ll x, ll y) {
    return x >= 0 && x < n && y >= 0 && y < m && !visited[x][y] && grid[x][y] == '.';
}

```

```cpp
// DFS to mark all connected floor cells of one room
void dfs(int x, int y) {
    visited[x][y] = true;
    for (int d = 0; d < 4; ++d) {
        int nx = x + dx[d];
        int ny = y + dy[d];
        if (isValid(nx, ny))
            dfs(nx, ny);
    }
}

int main() {
    cin >> n >> m;
    grid.resize(n);
    visited.resize(n, vector<bool>(m, false));

    // Read the grid
    for (int i = 0; i < n; ++i)
        cin >> grid[i];

    // Traverse all cells to find unvisited rooms
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            if (!visited[i][j] && grid[i][j] == '.') {
                dfs(i, j);   // Explore the full room
                rooms++;     // Count one room
            }
        }
    }

    cout << rooms << "\n";
    return 0;
}
```

1.2. Labyrinth

Question - Labyrinth Backup Link

Explanation :

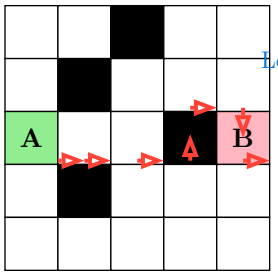
Concepts used: **BFS** (Breadth-First Search) - see Concepts

Given a grid map where `.` represents floor and `#` represents walls, we need to find the shortest path from cell `A` (start) to cell `B` (end). If a path exists, we must also output the actual path using directions: L (left), R (right), U (up), D (down).

Why BFS Works: BFS explores cells layer by layer based on distance from the source. Since all moves have equal cost (1 step), the first time we reach the destination is guaranteed to be via the shortest path. This is the key property that makes BFS optimal for unweighted shortest paths.

Algorithm:
1. Find positions of A and B in the grid
2. Start BFS from A, marking cells as visited
3. For each cell, try all 4 directions
4. Track the parent of each cell to reconstruct the path
5. When B is reached, backtrack through parents to build the path string

Example: Find path from A to B



Level 0: A
Level 1: neighbors of A
Level 2: ...
...

Path: RRRURDRD (length 8)
BFS guarantees this is the shortest!

BFS Exploration Order:

```
Step 0: Start at A (2,0)
        Queue: [(2,0)]

Step 1: Process (2,0), add neighbors
        Visit: (1,0), (2,1), (3,0)
        Queue: [(1,0), (2,1), (3,0)]

Step 2: Process neighbors, expand further
        Each level = 1 more step from start

Step N: First time we see B = shortest path found!
```

Key Implementation Details:

- Use a parent array to store where each cell was reached from
- Store both the parent position and the direction taken
- After reaching B, backtrack from B to A using parents
- Reverse the path string since we built it backwards

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<string> grid;
vector<vector<bool>> visited;
vector<vector<pair<int,int>>> parent;
vector<vector<char>> dir;

int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0};
char dc[] = {'R', 'L', 'D', 'U'};

bool isValid(int x, int y) {
    return x >= 0 && x < n && y >= 0 && y < m &&
           !visited[x][y] && grid[x][y] != '#';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> m;
    grid.resize(n);
    visited.resize(n, vector<bool>(m, false));
    parent.resize(n, vector<pair<int,int>>(m, {-1, -1}));
    dir.resize(n, vector<char>(m, ' '));

    int ax, ay, bx, by;
    for (int i = 0; i < n; i++) {
        cin >> grid[i];
        for (int j = 0; j < m; j++) {
            if (grid[i][j] == 'A') { ax = i; ay = j; }
            if (grid[i][j] == 'B') { bx = i; by = j; }
        }
    }

    // BFS
```

```
    queue<pair<int,int>> q;
    q.push({ax, ay});
    visited[ax][ay] = true;

    while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();

        if (x == bx && y == by) {
            // Reconstruct path
            string path = "";
            int cx = bx, cy = by;
            while (cx != ax || cy != ay) {
                path += dir[cx][cy];
                auto [px, py] = parent[cx][cy];
                cx = px; cy = py;
            }
            reverse(path.begin(), path.end());
            cout << "YES\n" << path.length() << "\n" << path << "\n";
            return 0;
        }

        for (int d = 0; d < 4; d++) {
            int nx = x + dx[d];
            int ny = y + dy[d];
            if (isValid(nx, ny)) {
                visited[nx][ny] = true;
                parent[nx][ny] = {x, y};
                dir[nx][ny] = dc[d];
                q.push({nx, ny});
            }
        }
    }

    cout << "NO\n";
    return 0;
}
```

1.3. Building Roads

Question - Building Roads Backup Link

Explanation :

Concepts used: **DFS** (for finding connected components) - see Concepts

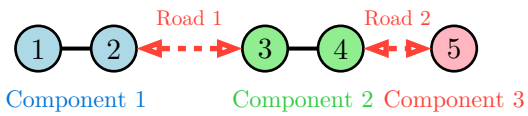
We have `n` cities and `m` existing roads. Some cities may not be connected to others. Our task is to find the minimum number of new roads needed to connect all cities, and output which cities should be connected.

Key Insight: The graph may consist of multiple disconnected components. To make the entire graph connected, we need to link these components together. If there are `k` connected components, we need exactly `k-1` new roads.

Algorithm:
1. Find all connected components using DFS/BFS
2. Pick one representative city from each component
3. Connect consecutive components: link component 1 to component 2, component 2 to component 3, etc.

Why k-1 Roads? Think of it like connecting islands with bridges. With k islands, you need k-1 bridges to form a single connected landmass. Each bridge reduces the number of separate components by 1.

Example: 5 cities, 2 roads: (1,2) and (3,4)



New roads needed: 2

Output: 2 → 3, 4 → 5

Any representative from each component works

Finding Connected Components:

```
DFS from city 1: visits {1, 2} → Component 1
DFS from city 3: visits {3, 4} → Component 2
DFS from city 5: visits {5}    → Component 3

Components found: 3
Roads needed: 3 - 1 = 2

Connect: 1→3 (or 2→3, 1→4, 2→4)
Connect: 3→5 (or 4→5)
```

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> component_rep;  // One node from each component

void dfs(int u) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v);
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> m;
    adj.resize(n + 1);
    visited.resize(n + 1, false);

    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    // Find all connected components
    for (int i = 1; i <= n; i++) {
        if (!visited[i]) {
            component_rep.push_back(i);  // Save one representative
            dfs(i);
        }
    }

    // Number of new roads = number of components - 1
    int roads_needed = component_rep.size() - 1;
    cout << roads_needed << "\n";

    // Connect consecutive components
```

```
46      for (int i = 0; i < roads_needed; i++) {
47          cout << component_rep[i] << " " << component_rep[i + 1] << "\n";
48      }
49
50      return 0;
51  }
```

1.4. Message Route

Question - Message Route Backup Link

Explanation :

Concepts used: **BFS** (shortest path in unweighted graphs) - see Concepts

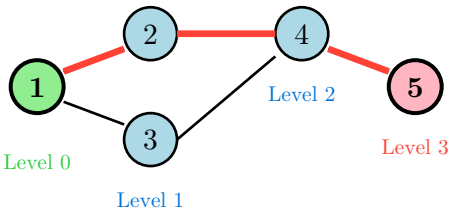
Syrjälä's network has `n` computers and `m` connections. We need to find the minimum number of computers a message must pass through to get from computer 1 to computer n, and output the actual route.

This is a classic shortest path problem in an unweighted graph. BFS is perfect here because it explores nodes level by level, guaranteeing the first path found to any node is the shortest.

Algorithm:
1. Start BFS from computer 1
2. For each computer, track which computer we came from (parent)
3. When we reach computer n, backtrack through parents to reconstruct the path
4. If computer n is never reached, output "IMPOSSIBLE"

Why BFS Guarantees Shortest Path: BFS visits all nodes at distance 1 first, then all nodes at distance 2, and so on. The first time we reach the destination, we've found it via the minimum number of edges.

Example: 5 computers, route from 1 to 5



Shortest path: 1 → 2 → 4 → 5

Length: 4 computers

BFS Trace:

```
Start: queue = [1], parent[1] = -1

Step 1: Process node 1
        Neighbors: 2, 3
        parent[2] = 1, parent[3] = 1
        queue = [2, 3]

Step 2: Process node 2
        Neighbors: 4
        parent[4] = 2
        queue = [3, 4]

```

```
Step 3: Process node 3
        Neighbors: 4 (already visited)
        queue = [4]

Step 4: Process node 4
        Neighbors: 5
        parent[5] = 4
        FOUND! Destination reached.

Backtrack: 5 ← 4 ← 2 ← 1
Path: 1 → 2 → 4 → 5
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    vector<int> parent(n + 1, -1);
    vector<bool> visited(n + 1, false);

    queue<int> q;
    q.push(1);
    visited[1] = true;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

```

```
        if (u == n) {
            // Reconstruct path
            vector<int> path;
            int curr = n;
            while (curr != -1) {
                path.push_back(curr);
                curr = parent[curr];
            }
            reverse(path.begin(), path.end());

            cout << path.size() << "\n";
            for (int node : path) {
                cout << node << " ";
            }
            cout << "\n";
            return 0;
        }

        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                parent[v] = u;
                q.push(v);
            }
        }
    }

    cout << "IMPOSSIBLE\n";
    return 0;
}
```

1.5. Building Teams

Question - Building Teams Backup Link

Explanation :

Concepts used: **BFS** (Graph coloring / bipartite checking) - see Concepts

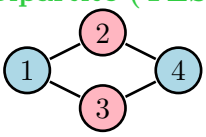
We have `n` pupils and `m` friendships. We need to divide all pupils into two teams such that no two friends are on the same team. This is the classic **bipartite graph checking** problem.

Key Insight: A graph can be divided into two groups with no edges within a group if and only if it is **bipartite**. A graph is bipartite if and only if it contains no odd-length cycles.

Algorithm (Graph Coloring with BFS/DFS):
1. Try to color each node with one of two colors (1 or 2)
2. Adjacent nodes must have different colors
3. If we ever try to color a node that's already colored with the opposite color, the graph is not bipartite
4. Process all connected components (the graph may be disconnected)

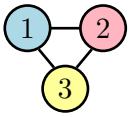
Bipartite Check: Can we 2-color the graph?

Bipartite (YES)



Team 1: {1, 4}
Team 2: {2, 3}

Not Bipartite (NO)



Odd cycle!
Node 3 needs a 3rd color

Coloring Process:

Start node 1 with color 1
Color all neighbors with color 2
Color their neighbors with color 1, etc.
Conflict = IMPOSSIBLE

BFS Coloring Trace:

```
Graph: 1-2, 2-3, 3-1 (triangle)

Start: color[1] = 1
Process 1: neighbor 2 → color[2] = 2
           neighbor 3 → color[3] = 2
Process 2: neighbor 1 (already colored 1, OK - different)
           neighbor 3 (already colored 2, OK - different)
Process 3: neighbor 1 (colored 1, we are 2 - OK)
           neighbor 2 (colored 2, we are 2 - CONFLICT!)

Result: IMPOSSIBLE (odd cycle detected)
```

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    vector<int> team(n + 1, 0);  // 0 = unvisited, 1 or 2 = team number
    bool possible = true;

    // Process all components
    for (int start = 1; start <= n && possible; start++) {
        if (team[start] != 0) continue;

        queue<int> q;
        q.push(start);
        team[start] = 1;

        while (!q.empty() && possible) {
            int u = q.front();
            q.pop();

            for (int v : adj[u]) {
                if (team[v] == 0) {
                    // Assign opposite team
                    team[v] = 3 - team[u];  // If u is 1, v is 2; if u is 2, v is 1
                    q.push(v);
                } else if (team[v] == team[u]) {
                    // Same team as neighbor - conflict!
                    possible = false;
                }
            }
        }
```

```
45      }
46
47      if (possible) {
48          for (int i = 1; i <= n; i++) {
49              cout << team[i] << " ";
50          }
51          cout << "\n";
52      } else {
53          cout << "IMPOSSIBLE\n";
54      }
55
56      return 0;
57  }
```

1.6. Round Trip

Question - Round Trip Backup Link

Explanation :

Concepts used: **DFS** (Cycle detection) - see Concepts

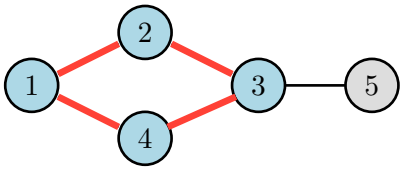
We need to find a route that starts and ends at the same city, visiting at least 3 cities. In graph terms, we need to find a **cycle** of length at least 3 in an undirected graph.

Key Insight: During DFS, if we encounter a node that is already visited and is not the immediate parent of the current node, we have found a cycle! The path from that node to the current node, plus the back edge, forms the cycle.

Algorithm:
1. Run DFS from any unvisited node
2. Track the parent of each node to avoid going back immediately
3. If we reach a visited node that isn't our parent, we found a cycle
4. Reconstruct the cycle by backtracking through parents

Important: In an undirected graph, going back to the parent doesn't count as a cycle (that's just the same edge traversed twice).

Cycle Detection with DFS



Cycle: 1 → 2 → 3 → 4 → 1

DFS from 1: visits 2, then 3, then 4
At 4: neighbor 1 is visited but not parent!
Back edge 4→1 reveals cycle

DFS Trace:

```
DFS(1): parent[1] = -1, visited[1] = true
  → DFS(2): parent[2] = 1, visited[2] = true
    → DFS(3): parent[3] = 2, visited[3] = true
      → DFS(4): parent[4] = 3, visited[4] = true
        → Check neighbor 1: visited AND not parent!
           CYCLE FOUND!

Backtrack from 4 to 1 using parents:
  4 → 3 → 2 → 1
Add the back edge to complete cycle:
  1 → 2 → 3 → 4 → 1
```

```
12
13 Output: 4 cities
14         1 2 3 4 1
```

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> parent;
int cycle_start = -1, cycle_end = -1;

bool dfs(int u, int p) {
    visited[u] = true;
    parent[u] = p;

    for (int v : adj[u]) {
        if (v == p) continue;  // Don't go back to parent

        if (visited[v]) {
            // Found a cycle!
            cycle_start = v;
            cycle_end = u;
            return true;
        }

        if (dfs(v, u)) return true;
    }
    return false;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> m;
    adj.resize(n + 1);
    visited.resize(n + 1, false);
    parent.resize(n + 1, -1);

```

```
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    // Try DFS from each unvisited node
    for (int i = 1; i <= n; i++) {
        if (!visited[i] && dfs(i, -1)) {
            // Reconstruct cycle
            vector<int> cycle;
            cycle.push_back(cycle_start);
            for (int curr = cycle_end; curr != cycle_start; curr =
            parent[curr]) {
                cycle.push_back(curr);
            }
            cycle.push_back(cycle_start);

            cout << cycle.size() << "\n";
            for (int node : cycle) {
                cout << node << " ";
            }
            cout << "\n";
            return 0;
        }
    }

    cout << "IMPOSSIBLE\n";
    return 0;
}
```

1.7. Monsters

Question - Monsters Backup Link

Explanation :

Concepts used: **BFS** (Multi-source BFS) - see Concepts

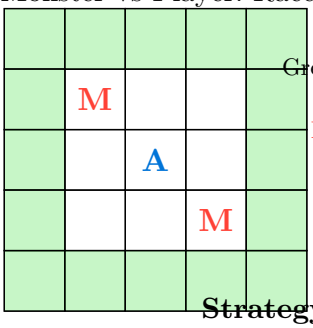
You're trapped in a labyrinth with monsters. Each turn, you and all monsters move simultaneously. You escape if you reach a boundary cell before any monster catches you (reaches the same cell at the same time or earlier).

Key Insight - Multi-source BFS: First, compute how quickly each monster can reach every cell using BFS starting from **all monsters simultaneously**. Then, BFS from your position, only moving to cells where you arrive strictly before any monster.

Algorithm:
1. **Monster BFS**: Start BFS from all monster positions at once (multi-source). This gives `monster_dist[i][j]` = minimum time for any monster to reach cell (i,j).
2. **Player BFS**: Start from player position. Only move to cell (nx, ny) if `player_time + 1 < monster_dist[nx][ny]`.
3. If player reaches any boundary cell, we have escaped!

Why Multi-source BFS? Instead of running separate BFS for each monster (slow), we add all monsters to the initial queue. This computes the minimum distance from **any** monster to each cell in a single pass.

Monster vs Player: Race to the Boundary



Green = Boundary
A = Player
M = Monster

Strategy:

1. Compute monster arrival times for all cells
2. Player BFS: only go where you beat monsters
3. Escape = reach boundary before monsters

Example Trace:

```
Grid:        Monster distances:    Player can escape?
. M .        . 0 1                 Check each boundary cell:
. A .   →    1 1 2                 - If player_dist < monster_dist: YES
. . M        2 2 0

Player at (1,1), needs to reach boundary
Option: Go UP to (0,1)
  Player arrives at time 1
```

```
  Monster arrives at time 1
  1 < 1? NO - monster catches us!

Option: Go LEFT to (1,0)
  Player arrives at time 1
  Monster arrives at time 1
  Still no good...

Need to find path where we're strictly faster!
```

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<string> grid;
int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0};
char dc[] = {'R', 'L', 'D', 'U'};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> m;
    grid.resize(n);

    int ax, ay;
    vector<pair<int,int>> monsters;

    for (int i = 0; i < n; i++) {
        cin >> grid[i];
        for (int j = 0; j < m; j++) {
            if (grid[i][j] == 'A') { ax = i; ay = j; }
            if (grid[i][j] == 'M') monsters.push_back({i, j});
        }
    }

    // Monster BFS - multi-source
    vector<vector<int>> monster_dist(n, vector<int>(m, INT_MAX));
    queue<pair<int,int>> q;

```

```
    for (auto [mx, my] : monsters) {
        monster_dist[mx][my] = 0;
        q.push({mx, my});
    }

    while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();
        for (int d = 0; d < 4; d++) {
            int nx = x + dx[d], ny = y + dy[d];
            if (nx >= 0 && nx < n && ny >= 0 && ny < m &&
                grid[nx][ny] != '#' && monster_dist[nx][ny] == INT_MAX) {
                monster_dist[nx][ny] = monster_dist[x][y] + 1;
                q.push({nx, ny});
            }
        }
    }

    // Player BFS
    vector<vector<int>> player_dist(n, vector<int>(m, INT_MAX));
    vector<vector<pair<int,int>>> parent(n, vector<pair<int,int>>(m, {-1,
    -1}));
    vector<vector<char>> move_dir(n, vector<char>(m, ' '));

    player_dist[ax][ay] = 0;
    q.push({ax, ay});

    while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();

        // Check if we escaped (boundary cell)
        if (x == 0 || x == n-1 || y == 0 || y == m-1) {
            // Reconstruct path
            string path = "";
            int cx = x, cy = y;
            while (cx != ax || cy != ay) {
                path += move_dir[cx][cy];
                auto [px, py] = parent[cx][cy];
                cx = px; cy = py;
            }
            reverse(path.begin(), path.end());
            cout << "YES\n" << path.length() << "\n" << path << "\n";
            return 0;
        }
```

```
76
77          for (int d = 0; d < 4; d++) {
78              int nx = x + dx[d], ny = y + dy[d];
79              if (nx >= 0 && nx < n && ny >= 0 && ny < m &&
80                  grid[nx][ny] != '#' && player_dist[nx][ny] == INT_MAX &&
81                  player_dist[x][y] + 1 < monster_dist[nx][ny]) {
82                  player_dist[nx][ny] = player_dist[x][y] + 1;
83                  parent[nx][ny] = {x, y};
84                  move_dir[nx][ny] = dc[d];
85                  q.push({nx, ny});
86              }
87          }
88      }
89
90      cout << "NO\n";
91      return 0;
92  }
```

1.8. Shortest Routes I

Question - Shortest Routes I Backup Link

Explanation :

Concepts used: **Dijkstra's Algorithm** - see Concepts

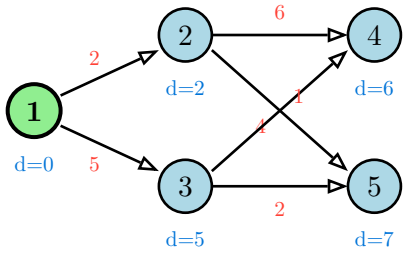
Find the shortest path from node 1 to all other nodes in a weighted directed graph. This is the classic **Dijkstra's algorithm** problem.

Why Not BFS? BFS works for unweighted graphs (all edges cost 1). With varying edge weights, BFS doesn't guarantee shortest paths. An edge of weight 10 should "cost more" than 10 edges of weight 1.

Dijkstra's Key Idea: Always process the unvisited node with the smallest known distance. Once processed, a node's distance is finalized (for non-negative weights). Use a priority queue to efficiently get the minimum.

Algorithm:
1. Initialize `dist[1] = 0`, all others = infinity
2. Add (0, 1) to priority queue (distance, node)
3. Extract minimum distance node, skip if already processed
4. For each neighbor, try to relax: if `dist[u] + weight < dist[v]`, update and add to queue
5. Repeat until queue is empty

Dijkstra's Algorithm: Shortest Paths from Node 1



Processing Order:

1 (d=0) → 2 (d=2) → 3 (d=5) → 4 (d=6) → 5 (d=7)

Dijkstra Trace:

```
Initial: dist = [0, ∞, ∞, ∞, ∞], PQ = [(0,1)]

Process node 1 (dist=0):
  → Relax 2: dist[2] = min(∞, 0+2) = 2
  → Relax 3: dist[3] = min(∞, 0+5) = 5
  PQ = [(2,2), (5,3)]

Process node 2 (dist=2):
  → Relax 4: dist[4] = min(∞, 2+6) = 8
  → Relax 5: dist[5] = min(∞, 2+4) = 6
```

```
  PQ = [(5,3), (6,5), (8,4)]

Process node 3 (dist=5):
  → Relax 4: dist[4] = min(8, 5+1) = 6  ← Better path!
  → Relax 5: dist[5] = min(6, 5+2) = 6  (no change)
  PQ = [(6,5), (6,4), (8,4)]

Process node 4 (dist=6): no outgoing edges

Process node 5 (dist=6): no outgoing edges

Final: dist = [0, 2, 5, 6, 6]
```

Time Complexity: O((n + m) log n) with priority queue.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = 1e18;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<pair<int, ll>>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int a, b;
        ll w;
        cin >> a >> b >> w;
        adj[a].push_back({b, w});
    }

    vector<ll> dist(n + 1, INF);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;

    dist[1] = 0;
    pq.push({0, 1});

```

```
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        // Skip if we've already found a better path
        if (d > dist[u]) continue;

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }

    for (int i = 1; i <= n; i++) {
        cout << dist[i] << " ";
    }
    cout << "\n";

    return 0;
}
```

1.9. Shortest Routes II

Question - Shortest Routes II Backup Link

Explanation :

Concepts used: **Floyd-Warshall Algorithm** (all-pairs shortest path) - see Concepts

Given a weighted graph, answer multiple queries: "What is the shortest path from node a to node b?" This is the **all-pairs shortest path** problem, solved efficiently with **Floyd-Warshall**.

Why Not Run Dijkstra for Each Query? With q queries and running Dijkstra each time, complexity is $O(q \cdot (n + m) \log n)$. For many queries, this is slow. Floyd-Warshall precomputes all pairs in $O(n^3)$, then answers each query in $O(1)$.

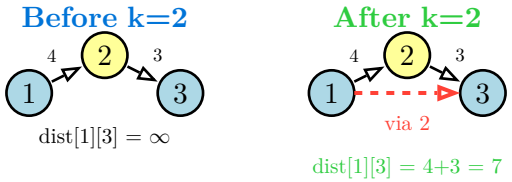
Floyd-Warshall Key Idea: Build up shortest paths by considering intermediate nodes one at a time. `dist[i][j]` through nodes {1, 2, ..., k} is either:

- The path not using k: `dist[i][j]` through {1..k-1}
- The path using k: `dist[i][k] + dist[k][j]` through {1..k-1}

Algorithm:

```
1  for k = 1 to n:          // Try each intermediate node
2    for i = 1 to n:        // For each source
3      for j = 1 to n:      // For each destination
4        dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

Floyd-Warshall: Adding Intermediate Nodes



Update Rule:

dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

For each k, check if going through k is better

Example Trace (3 nodes):

```
1  Initial distances:
2        1    2    3
3  1 [   0    4    ∞ ]
4  2 [   ∞    0    3 ]
5  3 [   2    ∞    0 ]
6
7  After k=1 (using node 1 as intermediate):
8    dist[3][2] = min(∞, dist[3][1] + dist[1][2]) = min(∞, 2+4) = 6
```

```
9
After k=2 (using node 2 as intermediate):
  dist[1][3] = min(∞, dist[1][2] + dist[2][3]) = min(∞, 4+3) = 7
  dist[3][1] = min(2, dist[3][2] + dist[2][1]) = min(2, 6+∞) = 2

After k=3 (using node 3 as intermediate):
  dist[1][2] = min(4, dist[1][3] + dist[3][2]) = min(4, 7+6) = 4
  dist[2][1] = min(∞, dist[2][3] + dist[3][1]) = min(∞, 3+2) = 5
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = 1e18;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, q;
    cin >> n >> m >> q;

    // Distance matrix
    vector<vector<ll>> dist(n + 1, vector<ll>(n + 1, INF));

    // Self-loops have distance 0
    for (int i = 1; i <= n; i++) {
        dist[i][i] = 0;
    }

    // Read edges (keep minimum if multiple edges)
    for (int i = 0; i < m; i++) {
        int a, b;
        ll c;
        cin >> a >> b >> c;
        dist[a][b] = min(dist[a][b], c);
        dist[b][a] = min(dist[b][a], c);
    }

    // Floyd-Warshall
    for (int k = 1; k <= n; k++) {
```

```
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (dist[i][k] < INF && dist[k][j] < INF) {
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
                }
            }
        }
    }

    // Answer queries
    while (q--) {
        int a, b;
        cin >> a >> b;
        if (dist[a][b] == INF) {
            cout << -1 << "\n";
        } else {
            cout << dist[a][b] << "\n";
        }
    }

    return 0;
}
```

1.10. High Score

Question - High Score Backup Link

Explanation :

Concepts used: **Bellman-Ford Algorithm** (with negative cycle detection) - see Concepts

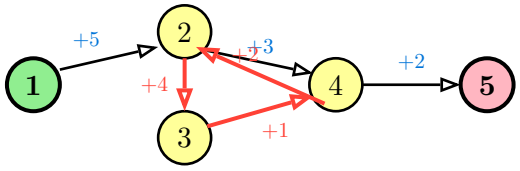
Find the maximum score when traveling from node 1 to node n. Each edge gives you points (can be negative). The twist: if you can get **infinite** score through a positive cycle reachable from 1 that can reach n, output -1.

Key Insight: This is longest path with cycle detection. We negate all weights and find shortest path, but we must detect **positive cycles** (which become negative cycles after negation) that lie on a path from 1 to n.

Bellman-Ford Algorithm:
- Relax all edges n-1 times to find shortest paths
- If we can still relax on the nth iteration, there's a negative cycle
- For this problem: we need cycles reachable from start AND that can reach end

Algorithm:
1. Run Bellman-Ford with negated weights (to find longest path)
2. After n-1 iterations, try n more relaxations
3. Any node relaxed in iterations n to 2n-1 is affected by a cycle
4. If node n is affected, answer is -1 (infinite score possible)

High Score: Longest Path with Positive Cycles



Positive cycle: 2→3→4→2 gains +7 each time!

Since cycle is reachable and reaches node 5...

Answer: −1 (infinite score possible)

Bellman-Ford Cycle Detection:

```
After n-1 rounds: dist[i] = shortest distance (with negated weights)

Round n to 2n-1: Keep relaxing
  If dist[v] can still be improved → v is affected by negative cycle

For High Score:
  1. Find all nodes reachable from node 1
  2. Find all nodes that can reach node n
  3. If any cycle-affected node is in both sets → answer is -1
```

```
4. Otherwise → answer is -dist[n] (negate back)
```

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = 1e18;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<tuple<int, int, ll>> edges;
    vector<vector<int>> adj(n + 1), radj(n + 1);

    for (int i = 0; i < m; i++) {
        int a, b;
        ll x;
        cin >> a >> b >> x;
        edges.push_back({a, b, -x});  // Negate for longest path
        adj[a].push_back(b);
        radj[b].push_back(a);
    }

    // Find nodes reachable from 1
    vector<bool> reach_from_1(n + 1, false);
    queue<int> q;
    q.push(1);
    reach_from_1[1] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (!reach_from_1[v]) {
                reach_from_1[v] = true;
                q.push(v);
            }
        }
    }
```

```
40
41      // Find nodes that can reach n
42      vector<bool> reach_to_n(n + 1, false);
43      q.push(n);
44      reach_to_n[n] = true;
45      while (!q.empty()) {
46          int u = q.front(); q.pop();
47          for (int v : radj[u]) {
48              if (!reach_to_n[v]) {
49                  reach_to_n[v] = true;
50                  q.push(v);
51              }
52          }
53      }
54
55      // Bellman-Ford
56      vector<ll> dist(n + 1, INF);
57      dist[1] = 0;
58
59      for (int i = 1; i <= n - 1; i++) {
60          for (auto [a, b, w] : edges) {
61              if (dist[a] < INF && dist[a] + w < dist[b]) {
62                  dist[b] = dist[a] + w;
63              }
64          }
65      }
66
67      // Check for negative cycles on path from 1 to n
68      for (int i = 1; i <= n; i++) {
69          for (auto [a, b, w] : edges) {
70              if (dist[a] < INF && dist[a] + w < dist[b]) {
71                  dist[b] = dist[a] + w;
72                  // If b is on a valid path, we have infinite score
73                  if (reach_from_1[b] && reach_to_n[b]) {
74                      cout << -1 << "\n";
75                      return 0;
76                  }
77              }
78          }
79      }
80
81      cout << -dist[n] << "\n";
82      return 0;
83  }
```

1.11. Flight Discount

Question - Flight Discount Backup Link

Explanation :

You can halve the cost of **exactly one** flight. Find the minimum total cost to travel from city 1 to city n. This is a classic **state-space Dijkstra** problem.

Key Insight - State Extension: Instead of just tracking distance to each node, track whether we've used the discount. State = (node, used_discount). This doubles our state space but keeps the problem solvable with Dijkstra.

Two Approaches:

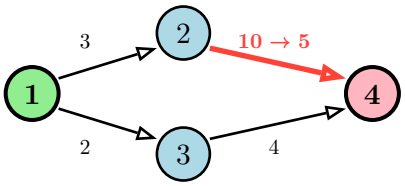
Approach 1: Two Dijkstra Runs

- Run Dijkstra from node 1: `dist1[v]` = shortest distance from 1 to v
- Run Dijkstra from node n on reversed graph: `dist2[v]` = shortest distance from v to n
- For each edge (u, v, w): try using discount on this edge
- Answer = min over all edges of: `dist1[u] + w/2 + dist2[v]`

Approach 2: Extended State Dijkstra

- State: (distance, node, used_discount)
- Transitions: regular edges, or discounted edge (if not used yet)

Flight Discount: Use Coupon on Best Edge



Without discount:

Path 1→2→4: cost = 3 + 10 = 13
Path 1→3→4: cost = 2 + 4 = 6

With discount on edge 2→4:

Path 1→2→4: cost = 3 + 5 = 8

Two-Dijkstra Approach:

```
dist1 from node 1:  [0, 3, 2, 6]   (1→2=3, 1→3=2, 1→3→4=6)
dist2 to node 4:    [6, 10, 4, 0]  (reverse: 4→3=4, 4→2=10, etc.)

For each edge, try discount:
  Edge 1→2 (w=3): dist1[1] + 3/2 + dist2[2] = 0 + 1 + 10 = 11
  Edge 1→3 (w=2): dist1[1] + 2/2 + dist2[3] = 0 + 1 + 4 = 5  ← Best!
  Edge 2→4 (w=10): dist1[2] + 10/2 + dist2[4] = 3 + 5 + 0 = 8
  Edge 3→4 (w=4): dist1[3] + 4/2 + dist2[4] = 2 + 2 + 0 = 4  ← Even better!

```

```
10 Answer: 4
```

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pli = pair<ll, int>;
const ll INF = 1e18;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<pair<int, ll>>> adj(n + 1), radj(n + 1);
    vector<tuple<int, int, ll>> edges;

    for (int i = 0; i < m; i++) {
        int a, b;
        ll c;
        cin >> a >> b >> c;
        adj[a].push_back({b, c});
        radj[b].push_back({a, c});
        edges.push_back({a, b, c});
    }

    // Dijkstra from node 1
    vector<ll> dist1(n + 1, INF);
    priority_queue<pli, vector<pli>, greater<>> pq;
    dist1[1] = 0;
    pq.push({0, 1});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d > dist1[u]) continue;
        for (auto [v, w] : adj[u]) {
            if (dist1[u] + w < dist1[v]) {
                dist1[v] = dist1[u] + w;
```

```
                pq.push({dist1[v], v});
            }
        }
    }

    // Dijkstra from node n on reversed graph
    vector<ll> dist2(n + 1, INF);
    dist2[n] = 0;
    pq.push({0, n});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d > dist2[u]) continue;
        for (auto [v, w] : radj[u]) {
            if (dist2[u] + w < dist2[v]) {
                dist2[v] = dist2[u] + w;
                pq.push({dist2[v], v});
            }
        }
    }

    // Try discount on each edge
    ll ans = INF;
    for (auto [a, b, c] : edges) {
        if (dist1[a] < INF && dist2[b] < INF) {
            ans = min(ans, dist1[a] + c / 2 + dist2[b]);
        }
    }

    cout << ans << "\n";
    return 0;
}
```

1.12. Cycle Finding

Question - Cycle Finding Backup Link

Explanation :

Concepts used: **Bellman-Ford Algorithm**, **Negative Cycle Detection** - see Concepts

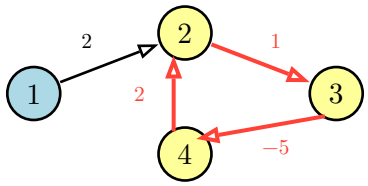
Find a negative cycle in a directed weighted graph, or report that none exists. A negative cycle is a cycle where the sum of edge weights is negative.

Why Negative Cycles Matter: In shortest path problems, negative cycles allow infinite reduction of path length by going around the cycle repeatedly. Detecting them is crucial for algorithms like Bellman-Ford.

Bellman-Ford for Cycle Detection:
- After n-1 relaxations, shortest paths are found (if no negative cycles)
- If we can still relax on the nth iteration, a negative cycle exists
- The node being relaxed in round n lies on (or is reachable from) a negative cycle

Reconstructing the Cycle:
- When we find a node x that can be relaxed in round n, follow parent pointers
- Go back n times to ensure we're inside the cycle (not just reachable from it)
- Then collect nodes until we return to the starting point

Negative Cycle Detection



Negative Cycle: 2 → 3 → 4 → 2

Sum: 1 + (-5) + 2 = −2 < 0

Each trip around reduces total by 2

Bellman-Ford Trace:

```
Edges: (1,2,2), (2,3,1), (3,4,-5), (4,2,2)

Round 1: dist = [0, 2, ∞, ∞]    (relax 1→2)
Round 2: dist = [0, 2, 3, ∞]    (relax 2→3)
Round 3: dist = [0, 2, 3, -2]   (relax 3→4)
Round 4: dist = [0, 0, 3, -2]   (relax 4→2: dist[4]+2 < dist[2])

Still can relax! dist[2] = 0 but...
Round 5: dist[3] = 0 + 1 = 1 < 3  (can still improve!)

Negative cycle detected!
```

```
12  Backtrack from node being relaxed to find cycle.
```

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = 1e18;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<tuple<int, int, ll>> edges;
    for (int i = 0; i < m; i++) {
        int a, b;
        ll c;
        cin >> a >> b >> c;
        edges.push_back({a, b, c});
    }

    vector<ll> dist(n + 1, 0);  // Start with 0 to detect all cycles
    vector<int> parent(n + 1, -1);
    int cycle_node = -1;

    // Bellman-Ford: n rounds
    for (int i = 1; i <= n; i++) {
        cycle_node = -1;
        for (auto [a, b, w] : edges) {
            if (dist[a] + w < dist[b]) {
                dist[b] = dist[a] + w;
                parent[b] = a;
                if (i == n) {
                    cycle_node = b;  // Found node on/reachable from negative cycle
                }
            }
        }
    }

```

```
    if (cycle_node == -1) {
        cout << "NO\n";
    } else {
        // Go back n times to ensure we're in the cycle
        for (int i = 0; i < n; i++) {
            cycle_node = parent[cycle_node];
        }

        // Collect the cycle
        vector<int> cycle;
        int curr = cycle_node;
        do {
            cycle.push_back(curr);
            curr = parent[curr];
        } while (curr != cycle_node);
        cycle.push_back(cycle_node);

        reverse(cycle.begin(), cycle.end());

        cout << "YES\n";
        for (int node : cycle) {
            cout << node << " ";
        }
        cout << "\n";
    }

    return 0;
}
```

1.13. Flight Routes

Question - Flight Routes Backup Link

Explanation :

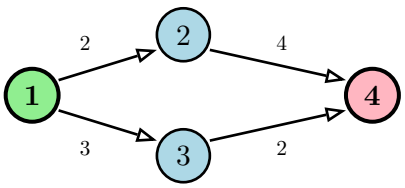
Find the k shortest route lengths from city 1 to city n. Routes can reuse cities and edges. This is the **k-shortest paths** problem.

Key Insight: Instead of stopping when we first reach node n (which gives the shortest path), we continue and collect the first k times we reach node n. Each arrival at n (possibly via different routes) gives us a candidate path length.

Modified Dijkstra for k-Shortest Paths:
- Allow each node to be visited up to k times
- Keep a count of how many times each node has been "finalized"
- When we pop a node from the priority queue, if it's been processed $<$ k times, process it
- Stop when node n has been processed k times

Why This Works: Dijkstra processes nodes in order of increasing distance. The first time we reach n is the shortest, the second time is the second shortest, etc. By allowing k visits per node, we find all k shortest paths.

K-Shortest Paths: Keep Processing Until k Arrivals



k=3 shortest paths to node 4:

1st: 1→3→4 = 3+2 = 5
2nd: 1→2→4 = 2+4 = 6
3rd: (need more edges for different path)

Algorithm Trace (k=3):

```
Priority Queue (min-heap): [(distance, node)]
count[i] = how many times node i has been finalized

Initial: PQ = [(0, 1)], count = [0, 0, 0, 0, 0]

Pop (0, 1): count[1] = 1
  Push: (2, 2), (3, 3)
  PQ = [(2, 2), (3, 3)]

Pop (2, 2): count[2] = 1
  Push: (6, 4)
  PQ = [(3, 3), (6, 4)]
```

```
13
14 Pop (3, 3): count[3] = 1
15   Push: (5, 4)
16   PQ = [(5, 4), (6, 4)]
17
18 Pop (5, 4): count[4] = 1  → Record: 1st shortest = 5
19 Pop (6, 4): count[4] = 2  → Record: 2nd shortest = 6
20 ...continue until count[n] = k
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pli = pair<ll, int>;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, k;
    cin >> n >> m >> k;

    vector<vector<pair<int, ll>>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int a, b;
        ll c;
        cin >> a >> b >> c;
        adj[a].push_back({b, c});
    }

    vector<int> count(n + 1, 0);
    vector<ll> result;

    priority_queue<pli, vector<pli>, greater<>> pq;
    pq.push({0, 1});

    while (!pq.empty() && count[n] < k) {
        auto [d, u] = pq.top();
        pq.pop();

        count[u]++;
```

```
33
34          if (u == n) {
35              result.push_back(d);
36              continue;
37          }
38
39          if (count[u] > k) continue;
40
41          for (auto [v, w] : adj[u]) {
42              pq.push({d + w, v});
43          }
44      }
45
46      for (ll dist : result) {
47          cout << dist << " ";
48      }
49      cout << "\n";
50
51      return 0;
52  }
```

1.14. Round Trip II

Question - Round Trip II Backup Link

Explanation :

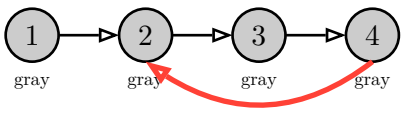
Find a cycle in a **directed** graph. Unlike Round Trip I (undirected), here edges have direction, making cycle detection different.

Key Difference from Undirected: In undirected graphs, any back edge creates a cycle. In directed graphs, we need a back edge to an **ancestor** in the current DFS path, not just any visited node.

Three Node States:
- **White (0)**: Unvisited
- **Gray (1)**: Currently being processed (on the current DFS path)
- **Black (2)**: Completely processed (all descendants explored)

Cycle Detection Rule: A cycle exists if and only if we find an edge from a gray node to another gray node. This means we found a back edge to an ancestor in our current path.

Directed Cycle Detection: Gray Node States



Back edge to gray node!

DFS Path: 1 → 2 → 3 → 4

Edge 4→2 goes to gray node (ancestor)

Cycle found: 2 → 3 → 4 → 2

DFS State Transitions:

```
DFS(1): color[1] = GRAY
  DFS(2): color[2] = GRAY
    DFS(3): color[3] = GRAY
      DFS(4): color[4] = GRAY
        Check neighbor 2: color[2] = GRAY
        → Back edge to ancestor! CYCLE FOUND!

Backtrack: 4 → 3 → 2
Cycle: 2 → 3 → 4 → 2
```

Code :

```cpp
#include <bits/stdc++.h>
```

```cpp
using namespace std;

int n, m;
vector<vector<int>> adj;
vector<int> color;  // 0=white, 1=gray, 2=black
vector<int> parent;
int cycle_start = -1, cycle_end = -1;

bool dfs(int u) {
    color[u] = 1;  // Gray

    for (int v : adj[u]) {
        if (color[v] == 1) {
            // Back edge to ancestor - cycle found!
            cycle_start = v;
            cycle_end = u;
            return true;
        }
        if (color[v] == 0) {
            parent[v] = u;
            if (dfs(v)) return true;
        }
    }

    color[u] = 2;  // Black
    return false;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> m;
    adj.resize(n + 1);
    color.resize(n + 1, 0);
    parent.resize(n + 1, -1);

    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
    }

    for (int i = 1; i <= n; i++) {
        if (color[i] == 0 && dfs(i)) {
```

```
            // Reconstruct cycle
            vector<int> cycle;
            cycle.push_back(cycle_start);
            for (int curr = cycle_end; curr != cycle_start; curr =
            parent[curr]) {
                cycle.push_back(curr);
            }
            cycle.push_back(cycle_start);

            reverse(cycle.begin(), cycle.end());

            cout << cycle.size() << "\n";
            for (int node : cycle) {
                cout << node << " ";
            }
            cout << "\n";
            return 0;
        }
    }

    cout << "IMPOSSIBLE\n";
    return 0;
}
```

1.15. Course Schedule

Question - Course Schedule Backup Link

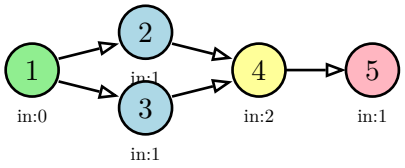
Explanation :

Given n courses and m requirements (course a must be completed before course b), find a valid order to complete all courses. This is **topological sorting**.

Key Property: A valid ordering exists if and only if the graph has no cycles (it's a DAG - Directed Acyclic Graph). In a topological order, for every edge u→v, node u appears before v.

Kahn's Algorithm (BFS-based):
1. Compute in-degree for each node (number of incoming edges)
2. Add all nodes with in-degree 0 to a queue (no prerequisites)
3. Process queue: remove node, add to result, decrease neighbors' in-degrees
4. If any neighbor's in-degree becomes 0, add it to queue
5. If result has all n nodes, we have a valid ordering; otherwise, cycle exists

Topological Sort: Process Zero In-degree Nodes



Processing Order:

1. Start with in-degree 0: node 1
2. Process 1 → nodes 2,3 become in-degree 0
3. Process 2,3 → node 4 becomes in-degree 0
4. Process 4 → node 5 becomes in-degree 0

Kahn's Algorithm Trace:

```
Initial in-degrees: [0, 1, 1, 2, 1]
Queue: [1] (only node with in-degree 0)
Result: []

Process 1: Result = [1]
  Decrease in-degree of 2, 3
  in-degrees: [0, 0, 0, 2, 1]
  Queue: [2, 3]

Process 2: Result = [1, 2]
  Decrease in-degree of 4
  in-degrees: [0, 0, 0, 1, 1]
  Queue: [3]
```

```
14
15 Process 3: Result = [1, 2, 3]
16   Decrease in-degree of 4
17   in-degrees: [0, 0, 0, 0, 1]
18   Queue: [4]
19
20 Process 4: Result = [1, 2, 3, 4]
21   Decrease in-degree of 5
22   Queue: [5]
23
24 Process 5: Result = [1, 2, 3, 4, 5]
25
26 All nodes processed → Valid topological order!
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<int>> adj(n + 1);
    vector<int> indegree(n + 1, 0);

    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        indegree[b]++;
    }

    // Kahn's algorithm
    queue<int> q;
    for (int i = 1; i <= n; i++) {
        if (indegree[i] == 0) {
            q.push(i);
        }
    }
```

```
28
29      vector<int> result;
30      while (!q.empty()) {
31          int u = q.front();
32          q.pop();
33          result.push_back(u);
34
35          for (int v : adj[u]) {
36              indegree[v]--;
37              if (indegree[v] == 0) {
38                  q.push(v);
39              }
40          }
41      }
42
43      if (result.size() != n) {
44          cout << "IMPOSSIBLE\n";
45      } else {
46          for (int node : result) {
47              cout << node << " ";
48          }
49          cout << "\n";
50      }
51
52      return 0;
53  }
```

1.16. Longest Flight Route

Question - Longest Flight Route Backup Link

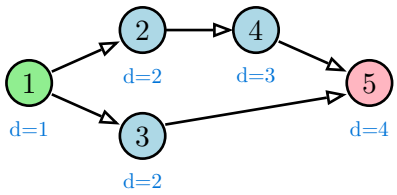
Explanation :

Find the longest path from city 1 to city n in a directed acyclic graph (DAG). Since it's a DAG, we can use dynamic programming with topological ordering.

Key Insight: In a DAG, we can process nodes in topological order. For each node, the longest path to it is 1 + max(longest path to any predecessor). This works because when we process a node, all its predecessors have already been processed.

Algorithm:
1. Topologically sort the graph
2. Initialize `dist[1] = 1` (path length includes starting node), others = -infinity
3. Process nodes in topological order: for each edge u→v, update `dist[v] = max(dist[v], dist[u] + 1)`
4. Track parent pointers to reconstruct the path

Longest Path in DAG: DP on Topological Order



Longest path: 1→2→4→5 (length 4)

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<int>> adj(n + 1), radj(n + 1);
    vector<int> indegree(n + 1, 0);

    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
```

```
        adj[a].push_back(b);
        radj[b].push_back(a);
        indegree[b]++;
    }

    // Topological sort
    queue<int> q;
    vector<int> topo;
    for (int i = 1; i <= n; i++) {
        if (indegree[i] == 0) q.push(i);
    }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        topo.push_back(u);
        for (int v : adj[u]) {
            if (--indegree[v] == 0) q.push(v);
        }
    }

    // DP for longest path
    vector<int> dist(n + 1, INT_MIN), parent(n + 1, -1);
    dist[1] = 1;

    for (int u : topo) {
        if (dist[u] == INT_MIN) continue;
        for (int v : adj[u]) {
            if (dist[u] + 1 > dist[v]) {
                dist[v] = dist[u] + 1;
                parent[v] = u;
            }
        }
    }

    if (dist[n] == INT_MIN) {
        cout << "IMPOSSIBLE\n";
    } else {
        vector<int> path;
        for (int cur = n; cur != -1; cur = parent[cur]) {
            path.push_back(cur);
        }
        reverse(path.begin(), path.end());
        cout << path.size() << "\n";
        for (int node : path) cout << node << " ";
        cout << "\n";
    }
```

```
62
63      return 0;
64  }
```

1.17. Game Routes

Question - Game Routes Backup Link

Explanation :

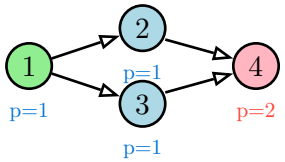
Count the number of paths from level 1 to level n in a directed acyclic graph. Since edges only go forward (DAG), we can use DP with topological ordering.

Key Insight: The number of paths to node v equals the sum of paths to all predecessors of v. Process in topological order so all predecessors are computed before the current node.

Recurrence:

```
paths[v] = sum of paths[u] for all edges u → v
paths[1] = 1 (one way to be at start)
```

Counting Paths: Sum from Predecessors



$$\text{paths}[4] = \text{paths}[2] + \text{paths}[3] = 1 + 1 = 2$$

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<int>> adj(n + 1);
    vector<int> indegree(n + 1, 0);

    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
```

```
        indegree[b]++;
    }

    // Topological sort
    queue<int> q;
    vector<int> topo;
    for (int i = 1; i <= n; i++) {
        if (indegree[i] == 0) q.push(i);
    }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        topo.push_back(u);
        for (int v : adj[u]) {
            if (--indegree[v] == 0) q.push(v);
        }
    }

    // Count paths
    vector<long long> paths(n + 1, 0);
    paths[1] = 1;

    for (int u : topo) {
        for (int v : adj[u]) {
            paths[v] = (paths[v] + paths[u]) % MOD;
        }
    }

    cout << paths[n] << "\n";
    return 0;
}
```

1.18. Investigation

Question - Investigation Backup Link

Explanation :

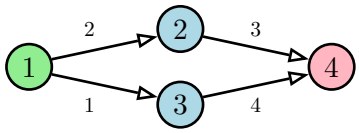
Find for the shortest route from 1 to n: (1) minimum price, (2) number of minimum-price routes, (3) minimum flights in a minimum-price route, (4) maximum flights in a minimum-price route.

Key Insight: Extend Dijkstra to track multiple values. When we find a shorter path, update all values. When we find an equal-length path, combine counts and update min/max flights.

State Updates:

- If `dist[u] + w < dist[v]`: found shorter path, reset all values
- If `dist[u] + w == dist[v]`: found another shortest path, add to count, update min/max flights

Extended Dijkstra: Track Multiple Properties



Two paths of cost 5: 1→2→4 and 1→3→4
count=2, min_flights=2, max_flights=2

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = 1e18;
const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<pair<int, ll>>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int a, b;
        ll c;
```

```
        cin >> a >> b >> c;
        adj[a].push_back({b, c});
    }

    vector<ll> dist(n + 1, INF);
    vector<ll> cnt(n + 1, 0);
    vector<int> minf(n + 1, 0), maxf(n + 1, 0);

    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
    dist[1] = 0;
    cnt[1] = 1;
    pq.push({0, 1});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        if (d > dist[u]) continue;

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                cnt[v] = cnt[u];
                minf[v] = minf[u] + 1;
                maxf[v] = maxf[u] + 1;
                pq.push({dist[v], v});
            } else if (dist[u] + w == dist[v]) {
                cnt[v] = (cnt[v] + cnt[u]) % MOD;
                minf[v] = min(minf[v], minf[u] + 1);
                maxf[v] = max(maxf[v], maxf[u] + 1);
            }
        }
    }

    cout << dist[n] << " " << cnt[n] << " " << minf[n] << " " << maxf[n] <<
    "\n";
    return 0;
}
```

1.19. Planets Queries I

Question - Planets Queries I Backup Link

Explanation :

Each planet has exactly one teleporter to another planet. Given queries "starting from planet x, where do you end up after k teleports?", answer efficiently using **binary lifting**.

Key Insight - Binary Lifting: Precompute `jump[i][x]` = planet reached from x after 2^i teleports. Any k can be decomposed into powers of 2, so we can answer queries by combining jumps.

Building the Table:

- `jump[0][x] = t[x]` (direct teleporter destination)
- `jump[i][x] = jump[i-1][jump[i-1][x]]` (jump 2^i = jump 2^{i-1} twice)

Binary Lifting: Precompute Power-of-2 Jumps

jump[0][x] = 1 step	
jump[1][x] = 2 steps	Query k=5 = 4+1:
jump[2][x] = 4 steps	Use jump[2] then jump[0]
...	

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

const int LOG = 30;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    vector<vector<int>> jump(LOG, vector<int>(n + 1));

    for (int i = 1; i <= n; i++) {
        cin >> jump[0][i];
    }

    // Build binary lifting table
    for (int j = 1; j < LOG; j++) {
        for (int i = 1; i <= n; i++) {
            jump[j][i] = jump[j-1][jump[j-1][i]];
```

```
        }
    }

    while (q--) {
        int x;
        long long k;
        cin >> x >> k;

        for (int j = 0; j < LOG; j++) {
            if (k & (1LL << j)) {
                x = jump[j][x];
            }
        }

        cout << x << "\n";
    }

    return 0;
}
```

1.20. Planets Queries II

Question - Planets Queries II Backup Link

Explanation :

Given queries "how many teleports to get from planet a to planet b?", find the distance or report impossible. This is more complex because the functional graph has cycles (rho-shaped).

Key Insight: Each connected component has exactly one cycle (since each node has out-degree 1). Find which cycle each node belongs to, distance to cycle, and position in cycle.

Cases:
1. If a and b are in different components: impossible
2. If b is on the path from a (before cycle): direct distance
3. If both reach the same cycle: distance to cycle + cycle traversal

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

const int LOG = 30;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    vector<int> t(n + 1);
    vector<vector<int>> jump(LOG, vector<int>(n + 1));

    for (int i = 1; i <= n; i++) {
        cin >> t[i];
        jump[0][i] = t[i];
    }

    for (int j = 1; j < LOG; j++) {
        for (int i = 1; i <= n; i++) {
            jump[j][i] = jump[j-1][jump[j-1][i]];
        }
    }

    // Find cycles and distances
```

```
    vector<int> cycle_id(n + 1, 0), pos_in_cycle(n + 1, -1), dist_to_cycle(n + 1, -1);
    vector<int> cycle_len;
    int num_cycles = 0;

    vector<int> visited(n + 1, 0), in_stack(n + 1, 0);

    for (int start = 1; start <= n; start++) {
        if (visited[start]) continue;

        vector<int> path;
        int cur = start;

        while (!visited[cur] && !in_stack[cur]) {
            in_stack[cur] = 1;
            path.push_back(cur);
            cur = t[cur];
        }

        if (in_stack[cur]) {
            // Found new cycle
            num_cycles++;
            cycle_len.push_back(0);
            int cycle_start = cur;
            int pos = 0;
            do {
                cycle_id[cur] = num_cycles;
                pos_in_cycle[cur] = pos++;
                dist_to_cycle[cur] = 0;
                cur = t[cur];
            } while (cur != cycle_start);
            cycle_len[num_cycles - 1] = pos;
        }

        // Mark path nodes
        for (int node : path) {
            in_stack[node] = 0;
            visited[node] = 1;
        }
    }

    // Compute dist_to_cycle for non-cycle nodes
    function<void(int)> compute_dist = [&](int u) {
        if (dist_to_cycle[u] != -1) return;
        compute_dist(t[u]);
```

```
        dist_to_cycle[u] = dist_to_cycle[t[u]] + 1;
        cycle_id[u] = cycle_id[t[u]];
    };

    for (int i = 1; i <= n; i++) {
        compute_dist(i);
    }

    while (q--) {
        int a, b;
        cin >> a >> b;

        if (cycle_id[a] != cycle_id[b]) {
            cout << -1 << "\n";
            continue;
        }

        // Check if b is reachable from a
        // Try walking from a to see if we hit b before cycle
        int steps = 0;
        int cur = a;
        bool found = false;

        while (dist_to_cycle[cur] > dist_to_cycle[b]) {
            cur = t[cur];
            steps++;
        }

        if (cur == b) {
            cout << steps << "\n";
            continue;
        }

        if (dist_to_cycle[a] > 0 && dist_to_cycle[b] > 0) {
            cout << -1 << "\n";
            continue;
        }

        if (dist_to_cycle[b] > 0) {
            cout << -1 << "\n";
            continue;
        }

        // Both on cycle or a reaches cycle then b
        steps = dist_to_cycle[a];
```

```
        cur = a;
        for (int i = 0; i < steps; i++) cur = t[cur];

        int clen = cycle_len[cycle_id[a] - 1];
        int diff = (pos_in_cycle[b] - pos_in_cycle[cur] + clen) % clen;
        cout << steps + diff << "\n";
    }

    return 0;
}
```

1.21. Planets Cycles

Question - Planets Cycles Backup Link

Explanation :

For each planet, find how many teleports until you return to a previously visited planet. This is finding the "tail + cycle length" for each node in a functional graph.

Key Insight: Starting from any node, you'll eventually enter a cycle. The answer is the distance to the cycle plus the cycle length.

Algorithm:
1. Find all cycles using DFS with timestamp tracking
2. For cycle nodes: answer = cycle length
3. For tail nodes: answer = distance to cycle + cycle length

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<int> t(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> t[i];
    }

    vector<int> ans(n + 1, 0);
    vector<int> visited(n + 1, 0), in_path(n + 1, 0), path_pos(n + 1, 0);

    for (int start = 1; start <= n; start++) {
        if (ans[start] > 0) continue;

        vector<int> path;
        int cur = start;

        while (!visited[cur] && !in_path[cur]) {
            in_path[cur] = 1;
            path_pos[cur] = path.size();
            path.push_back(cur);
```

```
            cur = t[cur];
        }

        int cycle_len = 0, cycle_start_pos = 0;

        if (in_path[cur]) {
            // Found cycle
            cycle_start_pos = path_pos[cur];
            cycle_len = path.size() - cycle_start_pos;

            // Set answer for cycle nodes
            for (int i = cycle_start_pos; i < (int)path.size(); i++) {
                ans[path[i]] = cycle_len;
            }
        }

        // Set answer for tail nodes (and cycle nodes if already visited)
        for (int i = (int)path.size() - 1; i >= 0; i--) {
            if (ans[path[i]] == 0) {
                ans[path[i]] = ans[t[path[i]]] + 1;
            }
            in_path[path[i]] = 0;
            visited[path[i]] = 1;
        }
    }

    for (int i = 1; i <= n; i++) {
        cout << ans[i] << " ";
    }
    cout << "\n";

    return 0;
}
```

1.22. Road Reparation

Question - Road Reparation Backup Link

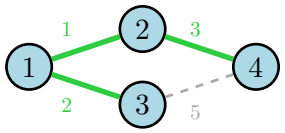
Explanation :

Concepts used: **Minimum Spanning Tree**, **DSU** (Disjoint Set Union for Kruskal's) - see Concepts

Find the minimum cost to connect all cities. This is the **Minimum Spanning Tree (MST)** problem.

Kruskal's Algorithm:
1. Sort all edges by weight
2. Use Union-Find (DSU) to track connected components
3. For each edge (in sorted order), if it connects two different components, add it to MST
4. Stop when we have n-1 edges (all nodes connected)

Kruskal's MST: Add Cheapest Edges First



MST cost = 1 + 2 + 3 = 6

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

vector<int> parent, rank_;

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}

bool unite(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return false;
    if (rank_[a] < rank_[b]) swap(a, b);
    parent[b] = a;
    if (rank_[a] == rank_[b]) rank_[a]++;
    return true;
```

```cpp
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    parent.resize(n + 1);
    rank_.resize(n + 1, 0);
    for (int i = 1; i <= n; i++) parent[i] = i;

    vector<tuple<ll, int, int>> edges;
    for (int i = 0; i < m; i++) {
        int a, b;
        ll c;
        cin >> a >> b >> c;
        edges.push_back({c, a, b});
    }

    sort(edges.begin(), edges.end());

    ll total = 0;
    int count = 0;

    for (auto [w, a, b] : edges) {
        if (unite(a, b)) {
            total += w;
            count++;
        }
    }

    if (count != n - 1) {
        cout << "IMPOSSIBLE\n";
    } else {
        cout << total << "\n";
    }

    return 0;
}
```

1.23. Road Construction

Question - Road Construction Backup Link

Explanation :

After each road is built, output the number of connected components and the size of the largest component. Use **Union-Find (DSU)** with size tracking.

Key Operations:

- `unite(a, b)`: merge components, update sizes
- Track component count (decreases by 1 on each successful merge)
- Track maximum component size

DSU: Track Components and Sizes

Initially: n components, each size 1
After merge: components–, update max size

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> parent, size_;

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}

void unite(int a, int b, int &components, int &max_size) {
    a = find(a); b = find(b);
    if (a == b) return;

    if (size_[a] < size_[b]) swap(a, b);
    parent[b] = a;
    size_[a] += size_[b];
    max_size = max(max_size, size_[a]);
    components--;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
```

```
    cin >> n >> m;

    parent.resize(n + 1);
    size_.resize(n + 1, 1);
    for (int i = 1; i <= n; i++) parent[i] = i;

    int components = n, max_size = 1;

    while (m--) {
        int a, b;
        cin >> a >> b;
        unite(a, b, components, max_size);
        cout << components << " " << max_size << "\n";
    }

    return 0;
}
```

1.24. Flight Routes Check

Question - Flight Routes Check Backup Link

Explanation :

Check if every city can reach every other city using flights. The graph is **strongly connected** if and only if this is true.

Key Insight: A directed graph is strongly connected iff:
1. Every node is reachable from node 1 (check with DFS/BFS from 1)
2. Node 1 is reachable from every node (check with DFS/BFS on reversed graph from 1)

If not strongly connected, find two nodes a, b where a cannot reach b.

Strongly Connected: Both DFS Must Visit All

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<int>> adj(n + 1), radj(n + 1);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        radj[b].push_back(a);
    }

    // BFS from node 1
    vector<bool> vis1(n + 1, false);
    queue<int> q;
    q.push(1);
    vis1[1] = true;
```

```
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (!vis1[v]) {
                vis1[v] = true;
                q.push(v);
            }
        }
    }

    // BFS from node 1 on reversed graph
    vector<bool> vis2(n + 1, false);
    q.push(1);
    vis2[1] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : radj[u]) {
            if (!vis2[v]) {
                vis2[v] = true;
                q.push(v);
            }
        }
    }

    // Check if all nodes visited in both
    for (int i = 1; i <= n; i++) {
        if (!vis1[i]) {
            cout << "NO\n" << 1 << " " << i << "\n";
            return 0;
        }
        if (!vis2[i]) {
            cout << "NO\n" << i << " " << 1 << "\n";
            return 0;
        }
    }

    cout << "YES\n";
    return 0;
}
```

1.25. Planets and Kingdoms

Question - Planets and Kingdoms Backup Link

Explanation :

Divide planets into kingdoms where planets in the same kingdom can reach each other. This is finding **Strongly Connected Components (SCCs)**.

Kosaraju's Algorithm:
1. DFS on original graph, push nodes to stack in finish order
2. DFS on reversed graph in stack order (reverse finish order)
3. Each DFS tree in step 2 is an SCC

Kosaraju: Two DFS Passes

1. DFS on G, record finish times
2. DFS on G 坦 in reverse finish order
3. Each tree = one SCC

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>> adj, radj;
vector<bool> visited;
vector<int> order, comp;
int num_scc = 0;

void dfs1(int u) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) dfs1(v);
    }
    order.push_back(u);
}

void dfs2(int u, int c) {
    comp[u] = c;
    for (int v : radj[u]) {
        if (comp[v] == 0) dfs2(v, c);
    }
}

int main() {
```

```
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> m;
    adj.resize(n + 1);
    radj.resize(n + 1);
    visited.resize(n + 1, false);
    comp.resize(n + 1, 0);

    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        radj[b].push_back(a);
    }

    // First DFS pass
    for (int i = 1; i <= n; i++) {
        if (!visited[i]) dfs1(i);
    }

    // Second DFS pass in reverse finish order
    reverse(order.begin(), order.end());
    for (int u : order) {
        if (comp[u] == 0) {
            num_scc++;
            dfs2(u, num_scc);
        }
    }

    cout << num_scc << "\n";
    for (int i = 1; i <= n; i++) {
        cout << comp[i] << " ";
    }
    cout << "\n";

    return 0;
}
```

1.26. Giant Pizza

Question - Giant Pizza Backup Link

Explanation :

Each person wants at least one of two toppings (+ means include, - means exclude). Find a valid assignment or report impossible. This is the classic **2-SAT** problem.

2-SAT Formulation:
- Variable x_i = true if topping i is included
- Clause $(a \vee b)$ becomes implication: $\neg a \Rightarrow b$ and $\neg b \Rightarrow a$

Solution via SCC:
- Build implication graph
- Find SCCs using Kosaraju's algorithm
- If x and $\neg x$ are in the same SCC, no solution exists
- Otherwise, for each variable, pick the literal whose SCC comes later in topological order

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>> adj, radj;
vector<int> order, comp;
vector<bool> visited;

void dfs1(int u) {
    visited[u] = true;
    for (int v : adj[u]) if (!visited[v]) dfs1(v);
    order.push_back(u);
}

void dfs2(int u, int c) {
    comp[u] = c;
    for (int v : radj[u]) if (comp[v] == -1) dfs2(v, c);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> m >> n;

    // 2n nodes: i for x_i, i+n for NOT x_i
```

```
    adj.resize(2 * n);
    radj.resize(2 * n);
    visited.resize(2 * n, false);
    comp.resize(2 * n, -1);

    for (int i = 0; i < m; i++) {
        char c1, c2;
        int t1, t2;
        cin >> c1 >> t1 >> c2 >> t2;
        t1--; t2--;

        // Convert to literal indices
        int a = (c1 == '+') ? t1 : t1 + n;
        int b = (c2 == '+') ? t2 : t2 + n;
        int na = (a < n) ? a + n : a - n;
        int nb = (b < n) ? b + n : b - n;

        // Add implications: NOT a => b, NOT b => a
        adj[na].push_back(b);
        adj[nb].push_back(a);
        radj[b].push_back(na);
        radj[a].push_back(nb);
    }

    // Kosaraju's algorithm
    for (int i = 0; i < 2 * n; i++) {
        if (!visited[i]) dfs1(i);
    }

    int num_scc = 0;
    reverse(order.begin(), order.end());
    for (int u : order) {
        if (comp[u] == -1) dfs2(u, num_scc++);
    }

    // Check satisfiability
    for (int i = 0; i < n; i++) {
        if (comp[i] == comp[i + n]) {
            cout << "IMPOSSIBLE\n";
            return 0;
        }
    }

    // Build assignment (pick literal with higher SCC number)
    for (int i = 0; i < n; i++) {
```

```
72          cout << (comp[i] > comp[i + n] ? '+' : '-') << " ";
73      }
74      cout << "\n";
75
76      return 0;
77  }
```

1.27. Coin Collector

Question - Coin Collector Backup Link

Explanation :

Collect maximum coins starting from any room. Rooms in the same SCC can all be collected together, so condense SCCs into a DAG, then find the longest path.

Algorithm:
1. Find SCCs and sum coins in each SCC
2. Build condensed DAG between SCCs
3. Find longest path in DAG using topological order DP

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int n, m;
vector<vector<int>> adj, radj;
vector<int> order, comp;
vector<bool> visited;
vector<ll> coins, scc_coins;

void dfs1(int u) {
    visited[u] = true;
    for (int v : adj[u]) if (!visited[v]) dfs1(v);
    order.push_back(u);
}

void dfs2(int u, int c) {
    comp[u] = c;
    scc_coins[c] += coins[u];
    for (int v : radj[u]) if (comp[v] == -1) dfs2(v, c);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> m;
    adj.resize(n + 1);
    radj.resize(n + 1);
```

```
    coins.resize(n + 1);
    visited.resize(n + 1, false);
    comp.resize(n + 1, -1);

    for (int i = 1; i <= n; i++) cin >> coins[i];

    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        radj[b].push_back(a);
    }

    // Find SCCs
    for (int i = 1; i <= n; i++) if (!visited[i]) dfs1(i);

    int num_scc = 0;
    scc_coins.resize(n + 1, 0);
    reverse(order.begin(), order.end());
    for (int u : order) if (comp[u] == -1) dfs2(u, num_scc++);

    // Build condensed DAG and find longest path
    vector<vector<int>> dag(num_scc);
    vector<int> indegree(num_scc, 0);

    for (int u = 1; u <= n; u++) {
        for (int v : adj[u]) {
            if (comp[u] != comp[v]) {
                dag[comp[u]].push_back(comp[v]);
                indegree[comp[v]]++;
            }
        }
    }

    // Topological sort + DP
    vector<ll> dp(num_scc);
    for (int i = 0; i < num_scc; i++) dp[i] = scc_coins[i];

    queue<int> q;
    for (int i = 0; i < num_scc; i++) {
        if (indegree[i] == 0) q.push(i);
    }

    while (!q.empty()) {
        int u = q.front(); q.pop();
```

```
76          for (int v : dag[u]) {
77              dp[v] = max(dp[v], dp[u] + scc_coins[v]);
78              if (--indegree[v] == 0) q.push(v);
79          }
80      }
81
82      cout << *max_element(dp.begin(), dp.end()) << "\n";
83      return 0;
84  }
```

1.28. Mail Delivery

Question - Mail Delivery Backup Link

Explanation :

Starting from post office 1, traverse every street exactly once and return to 1. This is finding an **Eulerian circuit** in an undirected graph.

Eulerian Circuit Exists When:
1. Graph is connected (considering only vertices with edges)
2. Every vertex has even degree

Hierholzer's Algorithm: Start from any vertex, greedily follow unused edges. When stuck, backtrack and insert the cycle. Use a stack-based implementation for efficiency.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<pair<int, int>>> adj(n + 1);
    vector<int> degree(n + 1, 0);
    vector<bool> used(m, false);

    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back({b, i});
        adj[b].push_back({a, i});
        degree[a]++;
        degree[b]++;
    }

    // Check if Eulerian circuit exists
    for (int i = 1; i <= n; i++) {
        if (degree[i] % 2 != 0) {
            cout << "IMPOSSIBLE\n";
            return 0;
        }
```

```
    }

    // Hierholzer's algorithm
    vector<int> path;
    stack<int> st;
    vector<int> ptr(n + 1, 0);
    st.push(1);

    while (!st.empty()) {
        int u = st.top();
        bool found = false;

        while (ptr[u] < (int)adj[u].size()) {
            auto [v, idx] = adj[u][ptr[u]++];
            if (!used[idx]) {
                used[idx] = true;
                st.push(v);
                found = true;
                break;
            }
        }

        if (!found) {
            path.push_back(u);
            st.pop();
        }
    }

    if ((int)path.size() != m + 1) {
        cout << "IMPOSSIBLE\n";
    } else {
        for (int node : path) cout << node << " ";
        cout << "\n";
    }

    return 0;
}
```

1.29. De Bruijn Sequence

Question - De Bruijn Sequence Backup Link

Explanation :

Find the shortest string containing all n-bit binary strings as substrings. Length is $2^n + n - 1$.

Key Insight: Model as Eulerian path problem. Create nodes for all (n-1)-bit strings. Edge from "abc" to "bcd" labeled with last bit. Finding Eulerian path gives De Bruijn sequence.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    if (n == 1) {
        cout << "01\n";
        return 0;
    }

    int nodes = 1 << (n - 1);
    int mask = nodes - 1;

    vector<int> ptr(nodes, 0);
    vector<int> path;
    stack<int> st;
    st.push(0);

    while (!st.empty()) {
        int u = st.top();
        if (ptr[u] < 2) {
            int v = ((u << 1) | ptr[u]++) & mask;
            st.push(v);
        } else {
            path.push_back(u);
            st.pop();
        }
```

```
    }

    reverse(path.begin(), path.end());

    string result(n - 1, '0');
    for (int i = 0; i < (int)path.size() - 1; i++) {
        result += ('0' + (path[i + 1] & 1));
    }

    cout << result << "\n";
    return 0;
}
```

1.30. Teleporters Path

Question - Teleporters Path Backup Link

Explanation :

Find a path using each teleporter exactly once, starting from level 1 and ending at level n. This is an **Eulerian path** in a directed graph.

Eulerian Path Exists When:
1. At most one vertex has out-degree - in-degree = 1 (start)
2. At most one vertex has in-degree - out-degree = 1 (end)
3. All other vertices have equal in-degree and out-degree
4. Graph is connected

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<int>> adj(n + 1);
    vector<int> in_deg(n + 1, 0), out_deg(n + 1, 0);

    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        out_deg[a]++;
        in_deg[b]++;
    }

    // Check Eulerian path conditions
    bool valid = true;
    valid &= (out_deg[1] - in_deg[1] == 1);  // Start at 1
    valid &= (in_deg[n] - out_deg[n] == 1);  // End at n

    for (int i = 2; i < n; i++) {
        if (in_deg[i] != out_deg[i]) valid = false;
    }
```

```
30
31      if (!valid) {
32          cout << "IMPOSSIBLE\n";
33          return 0;
34      }
35
36      // Hierholzer's algorithm
37      vector<int> path;
38      vector<int> ptr(n + 1, 0);
39      stack<int> st;
40      st.push(1);
41
42      while (!st.empty()) {
43          int u = st.top();
44          if (ptr[u] < (int)adj[u].size()) {
45              st.push(adj[u][ptr[u]++]);
46          } else {
47              path.push_back(u);
48              st.pop();
49          }
50      }
51
52      reverse(path.begin(), path.end());
53
54      if ((int)path.size() != m + 1) {
55          cout << "IMPOSSIBLE\n";
56      } else {
57          for (int node : path) cout << node << " ";
58          cout << "\n";
59      }
60
61      return 0;
62  }
```

1.31. Hamiltonian Flights

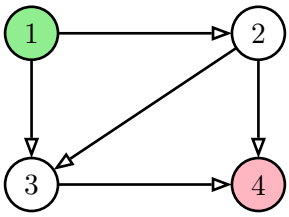
Question - Hamiltonian Flights Backup Link

Explanation :

Count the number of **Hamiltonian paths** from city 1 to city n. A Hamiltonian path visits every node exactly once.

Key Insight: Use **bitmask DP** where `dp[mask][i]` = number of paths that:
- Visit exactly the cities in `mask`
- End at city `i`

Algorithm:
1. State: `dp[mask][i]` where mask has bit j set if city j is visited
2. Base case: `dp[1][0] = 1` (only city 1 visited, at city 1)
3. Transition: For each edge (j → i), `dp[mask | (1<<i)][i] += dp[mask][j]`
4. Answer: `dp[(1<<n)-1][n-1]` (all cities visited, at city n)



Graph with 4 cities

Bitmask DP States:

`mask = 0001`: at city 1

`mask = 0011`: cities {1,2} visited

`mask = 0111`: cities {1,2,3} visited

`mask = 1111`: all visited, at city 4

Example path: 1→2→3→4

Another: 1→3→2→4

Figure 23: Hamiltonian path counting with bitmask DP

Trace Example (n=4):
- `dp[0001][0] = 1` (start at city 1)
- From city 1: go to 2 → `dp[0011][1] = 1`
- From city 1: go to 3 → `dp[0101][2] = 1`
- From city 2: go to 3 → `dp[0111][2] += dp[0011][1] = 1`
- From city 2: go to 4 → `dp[1011][3] = 1`
- From city 3: go to 4 → `dp[1111][3] += dp[0111][2] = 1`
- Continue until `dp[1111][3]` = total Hamiltonian paths to city 4

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```cpp

    int n, m;
    cin >> n >> m;

    vector<vector<int>> adj(n);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        a--; b--;
        adj[a].push_back(b);
    }

    // dp[mask][i] = number of Hamiltonian paths ending at i with visited set = mask
    vector<vector<long long>> dp(1 << n, vector<long long>(n, 0));
    dp[1][0] = 1;  // Start at city 1 (0-indexed: city 0)

    for (int mask = 1; mask < (1 << n); mask++) {
        for (int u = 0; u < n; u++) {
            if (!(mask & (1 << u))) continue;  // u must be in mask
            if (dp[mask][u] == 0) continue;

            for (int v : adj[u]) {
                if (mask & (1 << v)) continue;  // v must not be visited
                int newMask = mask | (1 << v);
                dp[newMask][v] = (dp[newMask][v] + dp[mask][u]) % MOD;
            }
        }
    }

    // Answer: all cities visited, ending at city n (0-indexed: n-1)
    cout << dp[(1 << n) - 1][n - 1] << "\n";

    return 0;
}
```

1.32. Knight's Tour

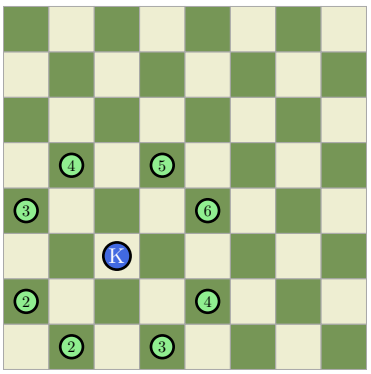
Question - Knight's Tour Backup Link

Explanation :

Find a knight's tour on an 8×8 chessboard starting from a given position. The knight must visit every square exactly once.

Key Insight: Use **backtracking** with **Warnsdorff's heuristic**: always move to the square with the fewest onward moves. This greedy heuristic dramatically reduces backtracking.

Algorithm:
1. From current position, find all valid knight moves
2. Sort moves by their **degree** (number of onward moves from each)
3. Try moves in order of ascending degree (Warnsdorff's rule)
4. If stuck, backtrack
5. Stop when all 64 squares are visited



Knight at (3,3): green = reachable squares
Numbers = degree (onward moves)
Warnsdorff: try degree 2 first

Figure 24: Warnsdorff's heuristic - prefer squares with fewer exits

Knight Move Offsets:

```
dx = {-2, -2, -1, -1, +1, +1, +2, +2}
dy = {-1, +1, -2, +2, -2, +2, -1, +1}
```

Why Warnsdorff Works:
- Visiting "corner" squares early (fewer exits) prevents getting trapped
- Leaves central squares (more exits) for later flexibility
- Almost always finds solution without backtracking

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;
```

```cpp

int board[8][8];
int dx[] = {-2, -2, -1, -1, 1, 1, 2, 2};
int dy[] = {-1, 1, -2, 2, -2, 2, -1, 1};

bool valid(int x, int y) {
    return x >= 0 && x < 8 && y >= 0 && y < 8 && board[x][y] == 0;
}

int degree(int x, int y) {
    int cnt = 0;
    for (int i = 0; i < 8; i++) {
        int nx = x + dx[i], ny = y + dy[i];
        if (valid(nx, ny)) cnt++;
    }
    return cnt;
}

bool solve(int x, int y, int move) {
    board[x][y] = move;
    if (move == 64) return true;

    // Get all valid next moves with their degrees
    vector<tuple<int, int, int>> next;
    for (int i = 0; i < 8; i++) {
        int nx = x + dx[i], ny = y + dy[i];
        if (valid(nx, ny)) {
            next.push_back({degree(nx, ny), nx, ny});
        }
    }

    // Sort by degree (Warnsdorff's heuristic)
    sort(next.begin(), next.end());

    for (auto& [deg, nx, ny] : next) {
        if (solve(nx, ny, move + 1)) return true;
    }

    board[x][y] = 0;  // Backtrack
    return false;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
48
49      int x, y;
50      cin >> x >> y;
51      x--; y--;  // Convert to 0-indexed
52
53      memset(board, 0, sizeof(board));
54
55      if (solve(y, x, 1)) {  // Note: input is (col, row)
56          for (int i = 0; i < 8; i++) {
57              for (int j = 0; j < 8; j++) {
58                  cout << board[i][j];
59                  if (j < 7) cout << " ";
60              }
61              cout << "\n";
62          }
63      }
64
65      return 0;
66  }
```

1.33. Download Speed

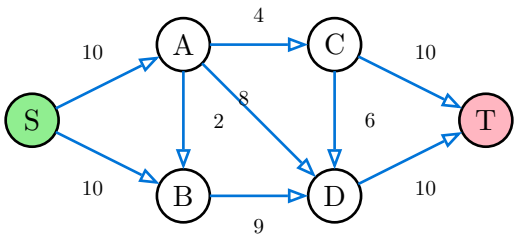
Question - Download Speed Backup Link

Explanation :

Find the **maximum flow** from computer 1 to computer n through a network of connections with limited capacities.

Key Insight: This is the classic **Maximum Flow** problem. Use **Edmonds-Karp algorithm** (BFS-based Ford-Fulkerson) or **Dinic's algorithm** for efficiency.

Dinic's Algorithm:
1. Build level graph using BFS (distance from source)
2. Find blocking flow using DFS (saturate paths)
3. Repeat until no augmenting path exists
4. Time: $O(V^2E)$ - much faster in practice



Max Flow = 19

Capacity on each edge

Figure 25: Network flow - find maximum flow from S to T

Key Concepts:
- **Residual Graph:** Shows remaining capacity on edges
- **Augmenting Path:** Path from S to T with positive residual capacity
- **Blocking Flow:** No more augmenting paths in level graph

Dinic's Algorithm Trace:
1. BFS builds level graph: S(0) → A,B(1) → C,D(2) → T(3)
2. DFS finds paths and pushes flow
3. Repeat until T unreachable from S

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int to, rev;
    long long cap;
};

```

```cpp
class Dinic {
    vector<vector<Edge>> graph;
    vector<int> level, iter;
    int n;

    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (auto& e : graph[v]) {
                if (e.cap > 0 && level[e.to] < 0) {
                    level[e.to] = level[v] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] >= 0;
    }

    long long dfs(int v, int t, long long f) {
        if (v == t) return f;
        for (int& i = iter[v]; i < (int)graph[v].size(); i++) {
            Edge& e = graph[v][i];
            if (e.cap > 0 && level[v] < level[e.to]) {
                long long d = dfs(e.to, t, min(f, e.cap));
                if (d > 0) {
                    e.cap -= d;
                    graph[e.to][e.rev].cap += d;
                    return d;
                }
            }
        }
        return 0;
    }

public:
    Dinic(int n) : n(n), graph(n), level(n), iter(n) {}

    void addEdge(int from, int to, long long cap) {
        graph[from].push_back({to, (int)graph[to].size(), cap});
        graph[to].push_back({from, (int)graph[from].size() - 1, 0});
    }
```

```
54
55      long long maxflow(int s, int t) {
56          long long flow = 0;
57          while (bfs(s, t)) {
58              fill(iter.begin(), iter.end(), 0);
59              long long f;
60              while ((f = dfs(s, t, LLONG_MAX)) > 0) {
61                  flow += f;
62              }
63          }
64          return flow;
65      }
66  };
67
68  int main() {
69      ios::sync_with_stdio(false);
70      cin.tie(nullptr);
71
72      int n, m;
73      cin >> n >> m;
74
75      Dinic dinic(n);
76      for (int i = 0; i < m; i++) {
77          int a, b;
78          long long c;
79          cin >> a >> b >> c;
80          a--; b--;
81          dinic.addEdge(a, b, c);
82      }
83
84      cout << dinic.maxflow(0, n - 1) << "\n";
85
86      return 0;
87  }
```

1.34. Police Chase

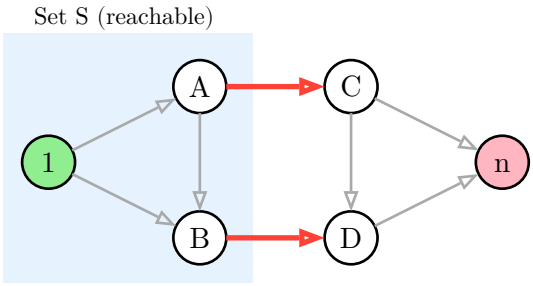
Question - Police Chase Backup Link

Explanation :

Find the **minimum number of streets** to block so the robber cannot reach crossroad n from crossroad 1. This is the **Minimum Cut** problem.

Key Insight: By the **Max-Flow Min-Cut theorem**, the minimum cut equals the maximum flow. After computing max flow, find the cut edges.

Finding the Min Cut:
1. Run max flow algorithm (Dinic's)
2. BFS from source in residual graph (only edges with remaining capacity)
3. Mark all reachable nodes as set S
4. Min cut edges: edges from S to non-S nodes that are saturated



Red edges = minimum cut

Block these streets!

Figure 26: Min cut separates source (node 1) from sink (node n)

Algorithm:
1. Build graph with capacity 1 for each undirected edge (add both directions)
2. Compute max flow from node 1 to node n
3. BFS from node 1 using only edges with remaining capacity > 0
4. For each original edge (u,v): if u is reachable and v is not, it's a cut edge

Why It Works:
- Max-Flow Min-Cut theorem guarantees optimality
- Saturated edges crossing the S/T partition form the minimum cut

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int to, rev;
    int cap;
};
```

```
8
9   class Dinic {
10  public:
11      vector<vector<Edge>> graph;
12      vector<int> level, iter;
13      int n;
14
15      bool bfs(int s, int t) {
16          fill(level.begin(), level.end(), -1);
17          queue<int> q;
18          level[s] = 0;
19          q.push(s);
20          while (!q.empty()) {
21              int v = q.front(); q.pop();
22              for (auto& e : graph[v]) {
23                  if (e.cap > 0 && level[e.to] < 0) {
24                      level[e.to] = level[v] + 1;
25                      q.push(e.to);
26                  }
27              }
28          }
29          return level[t] >= 0;
30      }
31
32      int dfs(int v, int t, int f) {
33          if (v == t) return f;
34          for (int& i = iter[v]; i < (int)graph[v].size(); i++) {
35              Edge& e = graph[v][i];
36              if (e.cap > 0 && level[v] < level[e.to]) {
37                  int d = dfs(e.to, t, min(f, e.cap));
38                  if (d > 0) {
39                      e.cap -= d;
40                      graph[e.to][e.rev].cap += d;
41                      return d;
42                  }
43              }
44          }
45          return 0;
46      }
47
48      Dinic(int n) : n(n), graph(n), level(n), iter(n) {}
49
50      void addEdge(int from, int to, int cap) {
51          graph[from].push_back({to, (int)graph[to].size(), cap});
```

```
52          graph[to].push_back({from, (int)graph[from].size() - 1, cap});  // Undirected
53      }
54
55      int maxflow(int s, int t) {
56          int flow = 0;
57          while (bfs(s, t)) {
58              fill(iter.begin(), iter.end(), 0);
59              int f;
60              while ((f = dfs(s, t, INT_MAX)) > 0) {
61                  flow += f;
62              }
63          }
64          return flow;
65      }
66  };
67
68  int main() {
69      ios::sync_with_stdio(false);
70      cin.tie(nullptr);
71
72      int n, m;
73      cin >> n >> m;
74
75      Dinic dinic(n);
76      vector<pair<int,int>> edges;
77
78      for (int i = 0; i < m; i++) {
79          int a, b;
80          cin >> a >> b;
81          a--; b--;
82          dinic.addEdge(a, b, 1);
83          edges.push_back({a, b});
84      }
85
86      int flow = dinic.maxflow(0, n - 1);
87
88      // Find reachable nodes from source in residual graph
89      vector<bool> reachable(n, false);
90      queue<int> q;
91      q.push(0);
92      reachable[0] = true;
93      while (!q.empty()) {
94          int u = q.front(); q.pop();
95          for (auto& e : dinic.graph[u]) {
```

```
96                if (e.cap > 0 && !reachable[e.to]) {
97                    reachable[e.to] = true;
98                    q.push(e.to);
99                }
100           }
101       }
102   
103       // Find cut edges
104       vector<pair<int,int>> cutEdges;
105       for (auto& [a, b] : edges) {
106           if ((reachable[a] && !reachable[b]) || (reachable[b] && !
          reachable[a])) {
107               cutEdges.push_back({a + 1, b + 1});
108           }
109       }
110   
111       cout << cutEdges.size() << "\n";
112       for (auto& [a, b] : cutEdges) {
113           cout << a << " " << b << "\n";
114       }
115   
116       return 0;
117   }
```

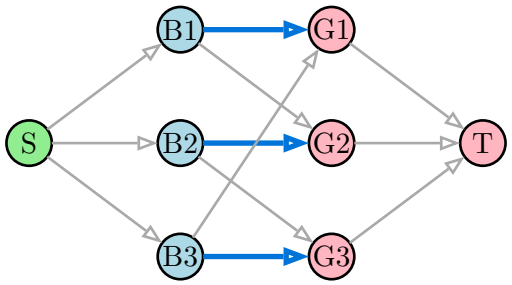
1.35. School Dance

Question - School Dance Backup Link

Explanation :

Find the **maximum matching** in a bipartite graph - pair maximum number of boys with girls where each person can only be in one pair.

Key Insight: Model as **maximum flow**:
- Source connects to all boys (capacity 1)
- Each boy connects to compatible girls (capacity 1)
- All girls connect to sink (capacity 1)
- Max flow = maximum matching



Blue = matched pairs

Max matching = 3

Figure 27: Bipartite matching as max flow

Alternative: Kuhn's Algorithm (Hungarian)

For bipartite matching, we can also use a simpler DFS-based approach:
- Try to match each boy with an available girl
- If girl is taken, try to reassign the current holder

Complexity: $O(V \times E)$ for Kuhn's algorithm, or $O(E\sqrt{V})$ using Hopcroft-Karp

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> adj[505];
int match_girl[505], match_boy[505];
bool used[505];
int n, m, k;

bool dfs(int boy) {
    for (int girl : adj[boy]) {
        if (used[girl]) continue;
        used[girl] = true;
```

```
13
14          // If girl is free OR we can reassign her current match
15          if (match_girl[girl] == -1 || dfs(match_girl[girl])) {
16              match_girl[girl] = boy;
17              match_boy[boy] = girl;
18              return true;
19          }
20      }
21      return false;
22  }
23
24  int main() {
25      ios::sync_with_stdio(false);
26      cin.tie(nullptr);
27
28      cin >> n >> m >> k;
29
30      for (int i = 0; i < k; i++) {
31          int a, b;
32          cin >> a >> b;
33          adj[a].push_back(b);
34      }
35
36      memset(match_girl, -1, sizeof(match_girl));
37      memset(match_boy, -1, sizeof(match_boy));
38
39      int matching = 0;
40      for (int boy = 1; boy <= n; boy++) {
41          memset(used, false, sizeof(used));
42          if (dfs(boy)) matching++;
43      }
44
45      cout << matching << "\n";
46      for (int boy = 1; boy <= n; boy++) {
47          if (match_boy[boy] != -1) {
48              cout << boy << " " << match_boy[boy] << "\n";
49          }
50      }
51
52      return 0;
53  }
```

1.36. Distinct Routes

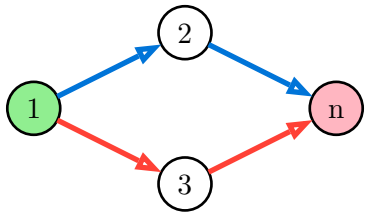
Question - Distinct Routes Backup Link

Explanation :

Find the **maximum number of edge-disjoint paths** from room 1 to room n, and output the paths.

Key Insight: This is max flow with capacity 1 on each edge. Each unit of flow represents one path, and capacity 1 ensures edges are not reused.

Algorithm:
1. Build flow network with capacity 1 on each edge
2. Compute max flow - this gives number of paths
3. Reconstruct paths by following saturated edges from source to sink



Path 1: $1 \to 2 \to n$

Path 2: $1 \to 3 \to n$

Max edge-disjoint paths $= 2$

Figure 28: Edge-disjoint paths - no edge shared between paths

Path Reconstruction: After max flow, for each path:
1. Start at node 1
2. Find an outgoing edge that was used (original capacity - residual > 0)
3. "Undo" the flow on that edge (restore capacity)
4. Move to next node, repeat until reaching n

Why Capacity 1:
- Each edge can only be used once
- Flow of k means k edge-disjoint paths exist

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int to, rev;
    int cap, flow;
};

```

```cpp
class Dinic {
public:
    vector<vector<Edge>> graph;
    vector<int> level, iter;
    int n;

    Dinic(int n) : n(n), graph(n), level(n), iter(n) {}

    void addEdge(int from, int to, int cap) {
        graph[from].push_back({to, (int)graph[to].size(), cap, 0});
        graph[to].push_back({from, (int)graph[from].size() - 1, 0, 0});
    }

    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (auto& e : graph[v]) {
                if (e.cap - e.flow > 0 && level[e.to] < 0) {
                    level[e.to] = level[v] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] >= 0;
    }

    int dfs(int v, int t, int f) {
        if (v == t) return f;
        for (int& i = iter[v]; i < (int)graph[v].size(); i++) {
            Edge& e = graph[v][i];
            if (e.cap - e.flow > 0 && level[v] < level[e.to]) {
                int d = dfs(e.to, t, min(f, e.cap - e.flow));
                if (d > 0) {
                    e.flow += d;
                    graph[e.to][e.rev].flow -= d;
                    return d;
                }
            }
        }
        return 0;
    }
```

```
54
55      int maxflow(int s, int t) {
56          int flow = 0;
57          while (bfs(s, t)) {
58              fill(iter.begin(), iter.end(), 0);
59              int f;
60              while ((f = dfs(s, t, INT_MAX)) > 0) {
61                  flow += f;
62              }
63          }
64          return flow;
65      }
66  };
67
68  int main() {
69      ios::sync_with_stdio(false);
70      cin.tie(nullptr);
71
72      int n, m;
73      cin >> n >> m;
74
75      Dinic dinic(n);
76      for (int i = 0; i < m; i++) {
77          int a, b;
78          cin >> a >> b;
79          a--; b--;
80          dinic.addEdge(a, b, 1);
81      }
82
83      int k = dinic.maxflow(0, n - 1);
84
85      cout << k << "\n";
86
87      // Reconstruct paths
88      for (int p = 0; p < k; p++) {
89          vector<int> path;
90          int cur = 0;
91          path.push_back(1);
92
93          while (cur != n - 1) {
94              for (auto& e : dinic.graph[cur]) {
95                  if (e.flow > 0) {
96                      e.flow--;
97                      path.push_back(e.to + 1);
98                      cur = e.to;
```

```
                    break;
                }
            }
        }

        cout << path.size() << "\n";
        for (int node : path) {
            cout << node << " ";
        }
        cout << "\n";
    }

    return 0;
}
```